

Programmation 1 – Épreuve de seconde chance

Durée : 2 heure.

Les documents manuscrits et dactylographiés sont autorisés. Les exercices sont indépendants. La clarté et la précision de la rédaction (en particulier les commentaires du code) seront un facteur important de l'évaluation. Le barème est donné à titre indicatif.

Exercice 1

Nous avons les classes suivantes :

```
1  /**
2   * La classe représentant les monuments historiques français.
3   *
4   * N.B. La fonction equals ne compare ici que les attributs
5   *     name et commune.
6   */
7  public class Monument {
8      public String type;           // type du monument
9      public String name;          // nom du monument
10     public int commune;           // code INSEE de la commune
11                                     // (entier à cinq chiffres, dont les 2 premiers
12                                     // correspondent au numéro de département)
13     public String address;        // adresse du monument
14     public String date;           // date de construction
15     public Monument(String type, String name, int commune,
16                       String address, String date) { ... }
17     ...
18 }
```

```
1  /**
2   * La classe représentant la base Mérimée des monuments historiques
3   * français. Les clefs sont des numéros des départements français
4   * (pour simplifier on ne considère que les numéros à deux chiffres),
5   * et les valeurs associées sont les ensembles (non vides) des
6   * monuments historiques présents dans ce département.
7   */
8  public class Merimee extends Table<Short, Set<Monument>>
```

Écrire les méthodes d'instance suivantes de la classe Merimee :

Question 1.1 (1,5 pts)

```
1  /**
2   * @param monument    monument historique
3   * @return             true si monument appartient à this, false sinon.
4   */
5  public boolean belongs(Monument monument)
```

Question 1.2 (2 pts)

```
1  /**
2   * @param address      adresse
3   * @return             true si un monument avec adresse address
4   *                   appartient à this, false sinon.
5   */
6  public boolean belongs(String address)
```

Question 1.3 (2 pts)

```
1  /**
2   * Supprimer de this tous les monuments situés dans la commune
3   * commune, détruite par une catastrophe naturelle.
4   *
5   * @param commune      code INSEE de la commune
6   */
7  public void remove(int commune)
```

Question 1.4 (2,5 pts)

```
1  /**
2   * Ajouter tous les monuments présents dans le tableau newMonuments à
3   * this. S'il y a déjà un monument du même nom dans la même commune,
4   * l'ancienne valeur est remplacée par la nouvelle valeur.
5   *
6   * @param newMonuments  table de monuments
7   * @pre                Chaque ligne du tableau correspond aux valeurs des
8   *                   5 attributs de la classe Monument, dans l'ordre.
9   *                   La troisième valeur est ainsi l'entier correspondant
10   *                   au code INSEE de la commune.
11   */
12 public void addAll(String[][] newMonuments)
```

Question 1.5 (3 pts)

```
1  /**
2   * @return la commune de this avec le plus de châteaux (type de
3   * monument ``château``) présents dans la base.
4   */
5  public int mostCastles()
```

Exercice 2

On s'intéresse à des arbres binaires à valeurs entières positives, entre 0 et 1000. Ces arbres sont organisés pour une recherche symétrique (comme un arbre binaire de recherche) relativement à ces valeurs entières.

Écrire les méthodes statiques suivantes :

Question 2.1 (1 pt)

```
1  /**
2   * En effectuant un parcours postfixe, afficher les nombres impairs
3   * présents dans l'arbre tree.
4   *
5   * @param    tree
6   */
7  public static void printValues(BinaryTree<Integer> tree)
```

Question 2.2 (2 pt)

```
1  /**
2   * @param    tree
3   * @param    depth
4   * @return    nombre de nœuds se trouvant dans tree
5   *            à une profondeur supérieure ou égale à depth
6   */
7  public static int countNodes(BinaryTree<Integer> tree, int depth)
```

Question 2.3 (3 pts)

```

1  /**
2   * @pre      !tree.isEmpty()
3   * @return   true si l'arbre tree est organisé de façon symétrique
4   *           (c'est-à-dire, la valeur d'un nœud est supérieure
5   *           aux valeurs dans son sous-arbre gauche, et inférieure
6   *           aux valeurs dans son sous-arbre droit), false sinon
7   */
8  public static boolean isSymmetric(BinaryTree<Integer> tree)

```

Question 2.4 (3 pt)

On considère les objets suivants :

```

1  public class MyPair {
2      public int value, count;
3      public MyPair () { value = count = 0; }
4      public MyPair (int value, int count) {
5          this.value = value;
6          this.count = count;
7      }
8      ...
9  }

```

```

1  /**
2   * @pre      !tree.isEmpty()
3   * @return   l'arbre binaire ayant la même structure que tree, mais
4   *           dans lequel les valeurs des nœuds sont des doublets
5   *           (value, count) où value est la même que celle de ce nœud
6   *           dans tree et count est le nombre de nœuds du sous-arbre
7   *           avec le nœud courant pour racine.
8   */
9  public static BinaryTree<MyPair> countTree(BinaryTree<Integer> tree)

```

Exemple :

